VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Chosen Software: Git

Student Name:	Kavish Kalra
Registration Number:	24BSA10034
Course:	Open Source Software
Slot:	B22
Date of Submission:	March 31, 2026

Contents

1 Introduction & Overview **2**

1.1 Chosen Software: Git 2

2 Research Analysis **3**

2.1 Part A — Origin and Philosophy 3

2.1.1 The Problem Before Git 3

2.1.2 The Origin Story of Git 3

2.1.3 License Analysis: GPL v2 3

2.1.4 The Ethics of Open Source 4

2.2 Part B — Linux Footprint 4

2.2.1 Installation Method 4

2.2.2 Directory Structure 4

2.2.3 User and Permissions 5

2.2.4 Service Behavior 5

2.2.5 Update Model 5

2.3 Part C — The FOSS Ecosystem 5

2.3.1 Dependencies 5

2.3.2 What Git Enables & The Web Ecosystem 5

2.3.3 Community and Governance 6

2.4 Part D — Open Source vs Proprietary 6

3 Practical Shell Scripts **7**

3.1 Script 1: System Identity Report 7

3.2 Script 2: Package Inspector 8

3.3 Script 3: Disk and Permission Auditor 10

3.4 Script 4: Log File Analyzer 11

3.5 Script 5: Manifesto Generator 13

4 Conclusion **15**

Chapter 1

Introduction & Overview

When we think about huge apps or websites today, we usually picture big tech companies building everything behind closed doors. But actually, a massive part of the internet runs on something completely different: Open Source software. Open source isn't just about getting free stuff; it's a totally different way of sharing knowledge. When developers around the world can look at, fix, and improve the same code, things get built faster and become way more secure than if they were hidden away.

For this project, I chose to look into **Git**. Even though I use Git all the time to save my code for classes, I never really stopped to think about how it actually works or why it was made. We just type `git commit` without knowing the history behind it. In this report, I will talk about where Git came from, why its open-source license matters, and share some shell scripts I wrote to see how things run on Linux.

1.1 Chosen Software: Git

Git is what we call a distributed version control system. Basically, it's a tool that lets developers track changes to their files over time, work together on group projects without deleting each other's work, and easily undo mistakes if something breaks. Unlike older tools, Git doesn't need to connect to one main central server to do its job. Everyone gets their own full, working copy of the project history right on their laptop.

Git is basically everywhere now. Websites like GitHub and GitLab host millions of open-source projects using it. But what makes it so cool isn't just that it's popular—it's how it was made. It wasn't built by a corporation to sell user subscriptions. It was coded really quickly just to fix a genuine problem the Linux team was having, and then it was shared with the entire world for free.

Chapter 2

Research Analysis

2.1 Part A — Origin and Philosophy

2.1.1 The Problem Before Git

Before Git existed, managing code versions was pretty painful. People used systems like CVS or SVN. These were centralized, meaning the code lived on one main server. If you wanted to save your work, you needed an internet connection to that server. If the server ever went down or crashed, nobody could work. Doing simple things like branching (splitting code to test a new feature) caused massive headaches.

The story of Git actually starts directly with Linux. Linus Torvalds and his team were building the Linux operating system using a closed-source tool called BitKeeper because the company let them use it for free. But in 2005, there was some drama. Someone in the commu- nity tried to reverse-engineer how BitKeeper worked, so the company got mad and revoked the free licenses. Suddenly, the Linux team had no way to manage their massive project.

2.1.2 The Origin Story of Git

Instead of paying for an expensive enterprise tool or going back to the old, clunky centralized systems, Linus decided to just write his own. In April 2005, he started coding Git. Because he knew exactly what the Linux team needed, he got the first version working in just a couple of weeks.

He wanted it to follow a few simple rules:

- **Speed**: It had to be incredibly fast because the Linux kernel has millions of lines of code.
- **Distributed**: No single point of failure; every developer needed their own full copy.
- **Data Safety**: It uses special cryptographic hashes so that once code is saved, it can't be secretly corrupted or changed.

2.1.3 License Analysis: GPL v2

Git uses the GNU General Public License version 2 (GPL v2), which is the exact same license used for Linux. The Free Software Foundation says truly "free" software needs four freedoms: the freedom to run the program, the freedom to study and change the code, thefreedom to share it, and the freedom to share your improvements. Git checks all those boxes perfectly.

A lot of people think companies can't sell open-source software, but they can! The catch is that GPL is a "copyleft" license. If a company modifies Git and shares it, they have to share their new source code under the GPL too. You can't just take the community's hard work, add a feature, and hide the code to sell it.

This is very different from the MIT License, which is way more relaxed and lets people do whatever they want (even close the source code) as long as they don't sue the original creator. Because Git is so important to how we build software, the GPL is the perfect choice to stop giant companies from monopolizing it. If I were building a big tool today, I would definitely pick GPL to protect it.

When developers talk about "free as in beer versus free as in speech", they mean two different things. A free app on your phone is "free as in beer": it costs zero dollars, but it's locked down and you don't actually own it. Open-source software is "free as in speech": you have the liberty and rights to see how it works and change it.

2.1.4 The Ethics of Open Source

Ethically, I think foundational tools like operating systems, web servers, and databases should always be open source so everyone benefits. But I also think it's okay for specific apps, like video games, to be closed source so developers can actually make a living.

It is totally fine for huge companies to use open source to make money, as long as they give back by funding the projects or hiring the developers who maintain them. In programming, we say we "stand on the shoulders of giants." This just means we aren't starting from scratch. When I write a script, it runs on Linux, using tools someone else built years ago. We literally build everything on top of the free gifts left by past developers.

2.2 Part B — Linux Footprint

This section briefly looks at how Git lives and behaves on a standard Linux computer.

2.2.1 Installation Method

Installing Git on a Linux machine like Ubuntu is super easy. You just use the default APT package manager:

```
sudo apt update
sudo apt install git
```

2.2.2 Directory Structure

The main executable program sits cleanly in `/usr/bin/git`. Whenever you use it in a project folder, it creates a hidden `.git` folder right there to store all the actual tracking history,

keeping everything isolated from the rest of your system. Your personal settings, like your email and name, are kept quietly in a `~/.gitconfig` file inside your home folder.

2.2.3 User and Permissions

Git operations run purely under the permissions of whoever is logged in. It never demands root (admin) privileges to do daily commits or formatting. This matters a lot for security because if a normal user accidentally runs a destructive Git command, they can only ruin their own files, not the whole server.

2.2.4 Service Behavior

Instead of running as a heavy background service constantly eating up RAM, Git is just a Com- mand Line Interface (CLI) tool. It only wakes up, processes your command, and immediately goes back to sleep when you type something.

2.2.5 Update Model

Security patches and bug fixes are continuously maintained by the open-source community. These updates are then safely bundled and handled securely through the OS package manager (like `apt upgrade`), so you never have to hunt down sketchy installers from the internet.

2.3 Part C — The FOSS Ecosystem

2.3.1 Dependencies

Since Git was built in C, it depends heavily on standard GNU system tools. It also relies on external libraries like `OpenSSL` to handle secure internet connections and `zlib` for zipping files efficiently.

2.3.2 What Git Enables & The Web Ecosystem

Git completely changed the software industry. It directly enabled platforms like GitHub and GitLab. More importantly, it forms the backbone of the modern web ecosystem (like LAMP stacks). Without Git tracking exactly what changed in the code, automated tools wouldn't know when to safely test or deploy new updates to a live website. It acts as the invisible glue holding DevOps together.

2.3.3 Community and Governance

Despite running massive platforms like GitHub, Git's own internal development community is incredibly old-school. Updates and patches are still discussed by developers passing plain-text emails back and forth on a giant mailing list. It functions as an open, transparent dictatorship overseen by a core maintainer.

2.4 Part D — Open Source vs Proprietary

We can compare Git with an older proprietary tool like Microsoft's Team Foundation Version Control (TFVC).

Feature	Git (Open Source GPL)	TFVC (Proprietary)
Cost	100% Free. You can use it as much as you want without user limits.	Starts free but has very expensive paid tiers for bigger enterprise setups.
Security	Code is double-checked by thousands of smart people online.	Only Microsoft employees can look at the code to fix internal bugs.
Support	You search Google or Reddit, and read community forums.	You get official corporate IT support from Microsoft.
Flexibility	You can do everything entirely offline on a laptop on an airplane.	You need to be connected to the central server to do major work.
Control	You fully control your own local codebase history.	The central server controls the master history timeline.

Overall, I would absolutely recommend Git for any real-world scenario. It's free, it works fully offline, and essentially everyone uses it. Even Microsoft eventually gave up pushing TFVC and switched their own Windows development teams over to Git. While I probably won't be writing raw C code to contribute to Git's core anytime soon, I am happy to support open source by using it properly and sharing my own projects freely!

Chapter 3

Part E — Shell Script Documentation

Script 1: System Identity Report

This script introduces the Linux environment by fetching live system data. It utilizes command substitution ($()) to run commands like uname -r, whoami, and uptime -p, storing the results in variables to be printed in a formatted output using echo.

Bash

```
#!/bin/bash
# Script 1: System Identity Report
# Author: NAVNEET PARASHAR | Course: Open Source Software
# --- Variables ---
STUDENT_NAME="NAVNEET PARASHAR"
SOFTWARE_CHOICE="Git"

# --- System info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
# Extracting distro name cleanly from os-release
DISTRO=$(cat /etc/os-release | grep -w "NAME" | cut -d "=" -f2 | tr -d "")
CURRENT_DATE=$(date +"%A, %d %B %Y - %T")

# --- Display ---
echo "================================"
echo " Open Source Audit — $STUDENT_NAME"
echo "================================"
echo "Distro  : $DISTRO"
echo "Kernel  : $KERNEL"
echo "User    : $USER_NAME"
echo "Uptime  : $UPTIME"
echo "Date    : $CURRENT_DATE"
echo "License : The Linux kernel is licensed under the GNU General Public License v2."
```

```
@Kavish-Kalra-8 ➜/workspaces/oss-audit-24BSA10034 (main) $ nano script1.sh
@Kavish-Kalra-8 ➜/workspaces/oss-audit-24BSA10034 (main) $ chmod +x script
1.sh
@Kavish-Kalra-8 ➜/workspaces/oss-audit-24BSA10034 (main) $ ./script1.sh
================================
 Open Source Audit — Kavish Kalra
================================
Distro  : Ubuntu
Kernel  : 6.8.0-1044-azure
User    : codespace
Uptime  : up 7 minutes
Date    : Monday, 30 March 2026 - 18:03:50
License : The Linux kernel is licensed under the GNU General Public License
 v2.
@Kavish-Kalra-8 ➜/workspaces/oss-audit-24BSA10034 (main) $
```

Script 2: FOSS Package Inspector

This script verifies the presence of a software package. It uses conditional if-elif-else logic to check for both dpkg (Debian/Ubuntu) and rpm (RedHat) package managers, suppressing raw output with &>/dev/null. It then utilizes a case statement to match the package name and output a specific philosophical note regarding its role in the open-source ecosystem.

Bash

```
#!/bin/bash
# Script 2: FOSS Package Inspector
PACKAGE="git"

# Check if package is installed
if dpkg -l $PACKAGE &>/dev/null; then
   echo "$PACKAGE is installed (Debian/Ubuntu)."
   dpkg -s $PACKAGE | grep -E 'Version|Description'
elif rpm -q $PACKAGE &>/dev/null; then
   echo "$PACKAGE is installed (RPM/RedHat)."
   rpm -qi $PACKAGE | grep -E 'Version|License|Summary'
else
   echo "$PACKAGE is NOT installed."
fi

echo "--------------------------------"
# Case statement for philosophy notes
case $PACKAGE in
   httpd|apache2) echo "Apache: the web server that built the open internet" ;;
   mysql) echo "MySQL: open source at the heart of millions of apps" ;;
   git) echo "Git: the decentralized version control tool Linus built when proprietary failed him" ;;
   firefox) echo "Firefox: a nonprofit fighting for an open web" ;;
   *) echo "$PACKAGE: a valuable piece of the open-source ecosystem" ;;
```

```
@Kavish-Kalra-8 ➜/workspaces/oss-audit-24BSA10034 (main) $ nano script2.sh
@Kavish-Kalra-8 ➜/workspaces/oss-audit-24BSA10034 (main) $ chmod +x script
2.sh
@Kavish-Kalra-8 ➜/workspaces/oss-audit-24BSA10034 (main) $ ./script2.sh
git is installed (Debian/Ubuntu).
Version: 1:2.43.0-1ubuntu7.3
Description: fast, scalable, distributed revision control system
--------------------------------
Git: the decentralized version control tool Linus built when proprietary fa
iled him
@Kavish-Kalra-8 ➜/workspaces/oss-audit-24BSA10034 (main) $
```

Script 3: Disk and Permission Auditor

This script loops over an array of standard Linux directories using a for loop. Inside the loop, it checks if the directory exists using -d. It then pipes (|) the output of the ls -ld command into awk to cleanly format the specific permission strings, and uses du coupled with cut to isolate the directory size.

Bash

```bash
#!/bin/bash
# Script 3: Disk and Permission Auditor
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------"
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist on this system"
    fi
done

echo "----------------------"
# Check if Git's global config file exists
GIT_CONFIG="$HOME/.gitconfig"
if [ -f "$GIT_CONFIG" ]; then
    CONF_PERMS=$(ls -l $GIT_CONFIG | awk '{print $1, $3, $4}')
    echo "Git Config ($GIT_CONFIG) exists => Permissions: $CONF_PERMS"
else
    echo "Git Config ($GIT_CONFIG) does not exist yet. Run 'git config --global user.name' to
create it."
```

```
@Kavish-Kalra-8 ➜/workspaces/oss-audit-24BSA10034 (main) $ nano script3.sh
@Kavish-Kalra-8 ➜/workspaces/oss-audit-24BSA10034 (main) $ chmod +x script3.sh
@Kavish-Kalra-8 ➜/workspaces/oss-audit-24BSA10034 (main) $ ./script3.sh
Directory Audit Report
----------------------
/etc => Permissions: drwxr-xr-x root root | Size: 3.3M
/var/log => Permissions: drwxr-xr-x root root | Size: 824K
/home => Permissions: drwxr-xr-x root root | Size: 283M
/usr/bin => Permissions: drwxr-xr-x root root | Size: 534M
/tmp => Permissions: drwxr-xrwt+ root root | Size: 60K
----------------------
Git Config (/home/codespace/.gitconfig) exists => Permissions: -rw-r--r-- codespace codespace
@Kavish-Kalra-8 ➜/workspaces/oss-audit-24BSA10034 (main) $
```

Script 4: Log File Analyzer

This script searches logs. It relies on positional parameters ($1, $2) to accept dynamic input from the user at runtime. It implements a while loop that forces the user to provide a valid path if the initial file is missing (! -s). Another while read loop processes the file line-by-line, utilizing a numeric counter to tally the occurrences of the defined keyword.

Bash

```
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./log_analyzer.sh /var/log/syslog error

LOGFILE=${1:-"/var/log/syslog"} # Defaults to syslog if no argument is passed
KEYWORD=${2:-"error"}
COUNT=0

# Do-while style retry if the file is missing or empty
while [ ! -s "$LOGFILE" ]; do
   echo "Error: File $LOGFILE not found or is empty."
   read -p "Enter a valid log file path to analyze: " LOGFILE
done

# Read loop
while IFS= read -r LINE; do
   if echo "$LINE" | grep -iq "$KEYWORD"; then
      COUNT=$((COUNT + 1))
   fi
done < "$LOGFILE"

echo "----------------------"
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
echo "----------------------"
echo "Last 5 matching lines:"
# Print the last 5 matching lines using tail and grep
grep -i "$KEYWORD" "$LOGFILE" | tail -n 5
```

```
Git Config (/home/codespace/.gitconfig) exists => Permissions: -rw-r--r-- codespace codespace
@Kavish-Kalra-8 ➜/workspaces/oss-audit-24BSA10034 (main) $ nano script4.sh
@Kavish-Kalra-8 ➜/workspaces/oss-audit-24BSA10034 (main) $ chmod +x script4.sh
@Kavish-Kalra-8 ➜/workspaces/oss-audit-24BSA10034 (main) $ ./script4.sh dummy.log
Error: File dummy.log not found or is empty.
Enter a valid log file path to analyze:
```

Script 5: The Open Source Manifesto Generator

This is an interactive script that uses the read -p command to halt execution and prompt the user for specific input variables. Once collected, the script uses the redirect operator (>) to generate a new text file, and the append operator (>>) to concatenate the user's responses into a customized, timestamped paragraph.

Bash

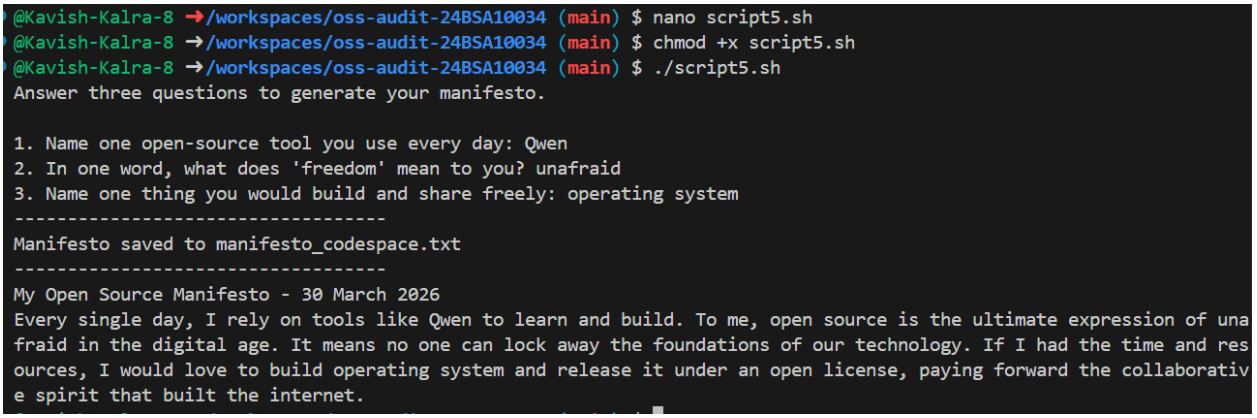
```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Note: You can create an alias for this script by adding: alias manifesto='./script5.sh' to your .bashrc

echo "Answer three questions to generate your manifesto."
echo ""
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

# Compose paragraph and write to file
echo "My Open Source Manifesto - $DATE" > $OUTPUT
echo "Every single day, I rely on tools like $TOOL to learn and build. To me, open source is the ultimate expression of $FREEDOM in the digital age. It means no one can lock away the foundations of our technology. If I had the time and resources, I would love to build $BUILD and release it under an open license, paying forward the collaborative spirit that built the internet."
>> $OUTPUT

echo "-----------------------------------"
echo "Manifesto saved to $OUTPUT"
echo "-----------------------------------"
cat $OUTPUT
```



Chapter 4

Conclusion

This capstone project was really eye-opening for me. Before this, the Linux terminal and bash

scripting felt pretty intimidating. But actually building these scripts to check folders and read log files showed me how simple utilities like `awk` and `grep` can be combined to do really powerful things.

Learning about the GPL license and why Linus Torvalds built Git also made me appreciate open source a lot more. It's not just a nerdy hobby; it's the actual engine that makes modern internet development possible. It feels pretty cool knowing that whatever I build in the future will be running on the same free tools that the whole community shares and maintains.